TIME & SPACE COMPLEXITY IN JAVA

A beginner-to-advanced guide for DSA, LeetCode, and technical interviews

Inside: Big O, loop rules, logarithms, recursion, auxiliary space, best/average/worst cases, Big O vs. Omega vs. Theta, amortized analysis, Java examples, and practice questions with answers.

Start from zero. Learn to reason about code instead of memorizing complexity labels.

How to use this guide

Read the chapters in order. For every code example, ask: (1) What is the input size? (2) How many times can the important operation run? (3) What extra memory is created?

Learning roadmap

- **Foundation:** algorithms and input size.
- **Time:** constant, linear, logarithmic, quadratic, and n log n.
- **Rules:** simplify expressions, add sequential work, analyze nested loops.
- **Space:** variables, arrays, collections, and recursion stack.
- **Advanced:** best/average/worst case, O/Ω/Θ, amortized analysis.
- **Practice:** DSA examples, mistakes, quiz, and answer key.

1. The foundation: what are we measuring?

What is an algorithm?

An algorithm is a sequence of steps that solves a problem. To find the largest number in an array, for example, inspect each element and keep track of the largest value seen so far.

What does n mean?

n usually means input size. If an array contains 8 numbers, n = 8. For two arrays, we may use n and m. For a tree, n often means the number of nodes.

Time complexity is not a stopwatch

Time complexity describes how the amount of work grows as input size grows. It does not predict exact seconds. Actual speed also depends on hardware, runtime, and implementation.

Space complexity

Space complexity describes how memory use grows. In interviews, **auxiliary space** often means extra memory used by the algorithm, excluding the input itself. Clarify which definition you use if it is ambiguous.

Key question: If input becomes 10 times larger, does the work stay roughly the same, grow 10 times, or grow around 100 times?

2. Big O: the most-used notation

Big O gives an asymptotic upper bound on growth. In beginner interview analysis, it is commonly used to describe an algorithm's dominant growth rate, often for the worst case. It ignores machine-specific timing and constant factors.

Complexity	Name	Typical example
O(1)	Constant	Array index access; fixed work
O(log n)	Logarithmic	Binary search; halve search range
O(n)	Linear	Visit every array element once
O(n log n)	Linearithmic	Merge sort
O(n^2)	Quadratic	Two n-sized nested loops
O(n^3)	Cubic	Three n-sized nested loops
O(2■)	Exponential	Some recursive subset enumeration
O(n!)	Factorial	Enumerate every permutation

For large inputs, a slower-growing complexity usually scales better. Constants, memory, and constraints still matter.

O(1): constant time

```
int[] numbers = {10, 20, 30, 40};
System.out.println(numbers[2]);
```

One array position is accessed directly. The work does not grow with the array length. Array indexing is O(1).

O(n): linear time

```
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}
```

The loop body runs n times. Doubling the number of elements roughly doubles the iterations. Time: O(n).

O(n²): quadratic time

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.println(i + j);
    }
}
```

For each of n outer iterations, the inner loop runs n times. Total executions = n × n = n². Time: O(n²).

O(log n): logarithmic time

```
int left = 0;
int right = sorted.length - 1;

while (left <= right) {
    int mid = left + (right - left) / 2;
    if (sorted[mid] == target) {
        return mid;
    } else if (sorted[mid] < target) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}
return -1;
```

Binary search requires a sorted array. Each iteration removes about half the candidates: n, n/2, n/4, n/8, ... The number of halvings is about log■(n), so time is O(log n).

O(n log n): linearithmic time

Merge sort divides the array into halves over about log n levels. Merging processes about n elements per level. Therefore, total work is n × log n = O(n log n).

3. Simplification rules

Rule 1: Ignore constant multipliers

O(5n) becomes O(n). O(100n) is also O(n). Big O focuses on growth, not the exact count of elementary operations.

Rule 2: Keep the dominant term

$O(n^2 + n + 10)$ becomes $O(n^2)$, because n^2 grows faster than n and a constant as n becomes large.

Expression	Simplified	Reason
7n + 20	O(n)	Linear term dominates constant
$n^2 + 5n + 1$	$O(n^2)$	Quadratic term dominates
$3n^3 + n^2$	$O(n^3)$	Cubic term dominates
n + log n	O(n)	n grows faster than log n
n log n + 4n	O(n log n)	n log n grows faster than n

Rule 3: Consecutive loops add

```
for (int i = 0; i < n; i++) {
    doWork();
}
for (int j = 0; j < n; j++) {
    doWork();
}
```

Each loop costs O(n). Total = O(n + n) = O(2n) = **O(n)**, not $O(n^2)$.

Rule 4: Nested loops often multiply

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        doWork();
    }
}
```

The inner loop runs n times for each outer iteration, so total work is $n \times n = O(n^2)$. Always inspect the loop bounds.

Rule 5: Different input sizes

```
for (int i = 0; i < n; i++) { doWork(); }
for (int j = 0; j < m; j++) { doWork(); }
```

Total is O(n + m). Do not simplify to O(n) unless there is a stated relationship between n and m. Nested loops sized n and m may be O(nm).

4. Common Java loop patterns

Pattern	Reasoning	Time
i++ until n	About n iterations	O(n)
i += 2 until n	About n/2; ignore constant	O(n)
i *= 2 until n	Value doubles each time	O(log n)
i /= 2 while positive	Value halves each time	O(log n)
Two n-sized nested loops	n × n body executions	$O(n^2)$
Nested loops sized n and m	n × m executions	O(nm)

Example: doubling loop

```
for (int i = 1; i < n; i *= 2) {
    System.out.println(i);
}
```

Values are 1, 2, 4, 8, 16, ... After k iterations, i is about 2^k. The loop stops near n, so k is about log■(n). Time: O(log n).

Example: triangular nested loop

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        doWork();
    }
}
```

The inner loop runs 0 + 1 + 2 + ... + (n - 1) times = n(n - 1)/2 = O(n²).

A nested loop is not automatically O(n²). If an inner pointer only moves forward through the array and never resets, total work may be O(n). Count actual total movements.

5. Space complexity in Java

Space analysis counts memory that grows with input size. Auxiliary space often excludes the input array but includes extra arrays, lists, maps, and recursion stack frames.

O(1) auxiliary space: fixed variables

```
public static int addFirstTwo(int[] numbers) {
    int first = numbers[0];
    int second = numbers[1];
    int sum = first + second;
    return sum;
}
```

Only a fixed number of variables are created. Auxiliary space: O(1).

O(n) auxiliary space: copy an array

```
public static int[] copyArray(int[] numbers) {
    int[] copy = new int[numbers.length];
    for (int i = 0; i < numbers.length; i++) {
        copy[i] = numbers[i];
    }
    return copy;
}
```

The loop takes O(n) time. The new array stores n elements, so auxiliary space is O(n).

O(n²) auxiliary space: a matrix

```
int[][] grid = new int[n][n];
```

The matrix has n × n = n² cells. Space: O(n²).

Printing versus storing

Printing every array item is O(n) time but O(1) auxiliary space if the algorithm does not store a growing collection. Output size and retained extra memory are different concepts.

6. Recursion: time and stack space

A recursive method calls itself. Each active call usually needs a stack frame. Stack space is a key part of space complexity.

Countdown example

```
public static void countdown(int n) {
    if (n <= 0) return;
    System.out.println(n);
    countdown(n - 1);
}
```

There are about n calls, each doing constant work: time O(n). At the deepest point, about n calls are active on the stack: auxiliary space O(n).

Factorial example

```
public static long factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

For n ≥ 1, there are about n calls, so time O(n). Recursion depth is O(n), so stack space is O(n). Very large inputs can overflow the stack.

Tree traversal

```
public static void preorder(TreeNode node) {
    if (node == null) return;
    System.out.println(node.val);
    preorder(node.left);
    preorder(node.right);
}
```

A traversal visits n nodes once: time O(n). Stack space depends on height h: O(h). A balanced tree has h = O(log n); a skewed tree can have h = O(n).

7. Best, average, and worst case

The same algorithm can do different amounts of work on different inputs of the same size.

Linear search

```
public static int linearSearch(int[] a, int target) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == target) return i;
    }
    return -1;
}
```

Case	Situation	Time
Best	Target is first element	O(1)
Average	Typical target position under a stated distribution	O(n)
Worst	Target is last or absent	O(n)

Average-case complexity depends on assumptions about input distribution. State assumptions when they matter.

8. Big O, Big Omega, and Big Theta

Notation	Meaning	Plain-English view
O(g(n))	Asymptotic upper bound	No faster than this growth rate, up to constants, eventually
Ω(g(n))	Asymptotic lower bound	At least this growth rate, up to constants, eventually
Θ(g(n))	Tight asymptotic bound	Matching upper and lower growth bounds

If T(n) = 3n + 5, then T(n) is O(n), Ω(n), and Θ(n). In casual DSA discussions people often say “Big O” to mean a growth class, but the formal definitions differ.

9. Amortized analysis: an advanced idea

Sometimes one operation is expensive but happens rarely. Amortized analysis analyzes the cost across a sequence of operations without depending on random inputs.

Dynamic array / ArrayList intuition

ArrayList append is O(1) amortized. When its internal array fills, it allocates a larger array and copies old elements, costing O(n) for that particular append. Since resizing happens infrequently as capacity grows geometrically, average cost per append across a long sequence is O(1) amortized.

Interview phrasing: “A single append can take O(n) during resizing, but append is O(1) amortized over a sequence of additions.”

10. Common algorithm complexity reference

Operation / algorithm	Typical time	Typical extra space
Array index access	O(1)	O(1)
Linear search	O(n)	O(1)
Binary search (sorted array)	O(log n)	O(1) iterative
Copy an array	O(n)	O(n)
Merge sort	O(n log n)	O(n) auxiliary array
Nested pair comparisons	$O(n^2)$	O(1) if no collection stored
HashMap get/put	O(1) average expected; may degrade in worst cases	Depends on entries
Tree traversal	O(n)	O(h) recursive stack

These are typical bounds, not universal guarantees. Details depend on implementation, data structure, and assumptions.

11. Interview checklist

- **Step 1:** Define input size: n, m, number of nodes, etc.
- **Step 2:** Identify the dominant operation.
- **Step 3:** Count iterations or recursive calls.
- **Step 4:** For sequential blocks, add; for genuinely nested loops, count the combined iterations.
- **Step 5:** Drop constant factors and lower-order terms.
- **Step 6:** Count extra arrays, collections, and recursion stack separately.
- **Step 7:** State best/average/worst case and assumptions if relevant.

Strong interview answer: “Let n be the array length. The loop visits each element once, so time is O(n). It uses only a fixed number of extra variables and creates no collection that grows with n, so auxiliary space is O(1).”

12. Common mistakes

- Saying every loop is O(n): a loop that doubles its variable is often O(log n).
- Saying two consecutive n-sized loops are $O(n^2)$: their total is O(n).
- Saying every nested loop is $O(n^2)$: inspect bounds and whether pointers reset.
- Forgetting recursion stack space.
- Calling extra space O(1) when the method builds a list of n items.
- Treating Big O as exact seconds.
- Ignoring input constraints: $O(n^2)$ may be fine for small n but impractical for huge n.
- Confusing total space with auxiliary space.

13. Practice quiz

Try the questions before checking the answer key. Let n be a positive input size.

Q1. One loop

```
for (int i = 0; i < n; i++) {
    System.out.println(i);
}
```

Choose: A) O(1) B) O(log n) C) O(n) D) $O(n^2)$

Q2. Doubling loop

```
for (int i = 1; i < n; i *= 2) {
    System.out.println(i);
}
```

Choose: A) O(1) B) O(log n) C) O(n) D) $O(n^2)$

Q3. Two consecutive loops

```
for (int i = 0; i < n; i++) { doWork(); }
for (int j = 0; j < n; j++) { doWork(); }
```

Choose: A) O(n) B) $O(n^2)$ C) O(log n) D) O(1)

Q4. Nested loops

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) { doWork(); }
}
```

Choose: A) O(n) B) $O(n^2)$ C) O(log n) D) O(1)

Q5. Sum an array

```
public static int findSum(int[] numbers) {
    int sum = 0;
    for (int i = 0; i < numbers.length; i++) {
        sum += numbers[i];
    }
    return sum;
}
```

Give the time and auxiliary space complexities.

Q6. Copy an array

A method creates a new array of n elements and copies all input elements. What are its time and auxiliary space complexities?

Q7. Tree traversal

A recursive traversal visits every node of a tree with n nodes and height h. What are its time and auxiliary stack-space complexities?

14. Quiz answer key

Q1: C — O(n)

The loop executes n times; constant work per iteration gives linear time.

Q2: B — O(log n)

The variable doubles: 1, 2, 4, 8, ... There are about log■(n) iterations.

Q3: A — O(n)

Sequential loops: O(n) + O(n) = O(2n) = O(n).

Q4: B — O(n²)

For each of n outer iterations, the inner loop runs n times: n × n = n².

Q5: Time O(n), auxiliary space O(1)

The loop visits each element once. Only a fixed number of extra variables are used.

Q6: Time O(n), auxiliary space O(n)

Copying n items requires n iterations, and the new array stores n elements.

Q7: Time O(n), auxiliary stack space O(h)

Every node is visited once. At most about h recursive calls are active along a root-to-leaf path.

15. Java mini-reference

```
One fixed operation:            O(1)
One pass over n items:           O(n)
Halve/double search range:       O(log n)
Two independent full passes:    O(n)
Two n-sized nested loops:       O(n^2)
Merge sort:                      O(n log n)
Store n extra items:             O(n) auxiliary space
Store an n-by-n matrix:           O(n^2) auxiliary space
Recursive tree traversal:        O(n) time, O(h) stack space
```

Avoid giving only a complexity label in interviews. Explain input size, count the work, and state what memory grows with input.

End of guide — analyze one Java method every day, explain your reasoning aloud, and then test yourself without looking at the answer key.